

SERVICE ACTUEL · DATASPHERE-FRONT

Interface de composition des labels SPIF

4 capacités analysées

services/datasphere-front · TypeScript/React/Vite, Chakra UI, Axios, XML DOMParser et keycloak-js.

Éditer et prévisualiser les classifications et catégories de sécurité associées à un dataset.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : Frontends, IAM, Catalog, Audit.

Périmètre de lecture

- Analyses existantes 11-ml-embeddings et 14-front-api reprises comme contexte ; vérifications ciblées des routes, outils et implémentations citées ci-dessous.
- Instructions racine et AGENTS.md du composant lorsqu'il existe ; aucun binaire propriétaire inspecté.
- Lecture statique ciblée pour finalisation ; les branches non exécutées et le déploiement restent non qualifiés.

Objets et cycle de vie

- Politiques SPIF, sélections, erreurs de validation, dataset et policy sauvegardée.

Dépendances du service

- Data-model et son proxy SPIF ; OIDC actuel Keycloak.

Fonctionnalités et compatibilité à préserver

DSPF-01 Cible non qualifiée

Charger la configuration SPIF et le dataset à labelliser

COMPORTEMENT OBSERVÉ

L'application extrait un UUID de dataset de l'URL /datasets/{id}, charge son nom et les politiques. Le chargement de configuration passe par un proxy de data-model /spif-proxy/files ; il parse un XML de configuration pour récupérer les URLs SPIF puis charge et parse chaque politique. Les erreurs de chargement empêchent l'affichage du formulaire.

UTILISATEURS ET RÔLES

Gestionnaires de labels.

INTERFACES DANS LE CODE

Route navigateur /datasets/{id} ; fetchConfig ; data-model /spif-proxy/files et /datasets/{id}.

DONNÉES ET ÉTATS

Dataset et politiques chargés dans l'état React.

DÉPENDANCES ET ÉCHANGES

Data-model, proxy de fichiers XML et parseur SPIF.

COMPORTEMENT CIBLE PROPOSÉ

Frontends TypeScript conserve ce parcours ; IAM/Catalog fournit une politique versionnée et une édition dataset explicite.

MIGRATION DE CETTE CAPACITÉ

Préserver formats XML et erreurs, migrer URLs backend et inclure version de politique dans la sauvegarde.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une politique XML invalide donne une erreur visible ; le nom du dataset et la politique proviennent du contexte autorisé.

Références de l'analyse

[monorepo](#) · [services/datasphere-front/app/src/App.tsx:9](#) Contexte dataset et chargement.

[monorepo](#) · [services/datasphere-front/app/src/lib/fetch-config.ts:29](#) Chargement XML via proxy.

DSPF-02 **Cible non qualifiée**

Composer une classification, catégories et marques de confidentialité**COMPORTEMENT OBSERVÉ**

Le formulaire choisit une politique et réinitialise les sélections au changement. Il restaure la policy enregistrée du dataset en retrouvant classification et catégories visibles. Les sélecteurs traitent classification, contexte, ensembles de catégories, texte libre, releasability et privacy marks. Les catégories cachées ou obsolètes influent sur les options affichées.

UTILISATEURS ET RÔLES

Auteurs de labels SPIF.

INTERFACES DANS LE CODE

SpifSelector ; ClassificationSelector ; ContextSelector ; CategorySelector ; ReleasabilitySelector ; PrivacyMarkSelector.

DONNÉES ET ÉTATS

LabelSelections : classificationLacv, categorySelections, freeTextSelections et privacyMarkSelections.

DÉPENDANCES ET ÉCHANGES

Modèle SPIF et policy actuelle du dataset.

COMPORTEMENT CIBLE PROPOSÉ

Frontend TS conserve la composition ; la décision serveur se fonde sur la même version de politique et des identifiants stables.

MIGRATION DE CETTE CAPACITÉ

Créer fixtures de politiques réelles anonymisées et comparer restauration des labels et changements de politique.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une sélection enregistrée est restituée à l'identique ; un changement de politique retire les sélections incompatibles sans sauvegarde implicite.

Références de l'analyse

[monorepo](#) · [services/datasphere-front/app/src/components/SpifSelector.tsx:33](#) Restauration depuis le label courant.

[monorepo](#) · [services/datasphere-front/app/src/components/SpifSelector.tsx:88](#) Réinitialisation et sélecteurs.

DSPF-03 **Cible non qualifiée****Appliquer les règles SPIF et prévisualiser le label****COMPORTEMENT OBSERVÉ**

spif-rules calcule exclusions, classifications admises, catégories requises et sélections effectives ; il détecte les conflits de contexte puis valide l'ensemble. Le formulaire calcule les issues et ne génère le label que sans erreur. Le preview affiche notamment les privacy marks et sérialise les groupes de catégories dans la syntaxe attendue par le backend. Cette validation navigateur n'est pas une autorisation serveur.

UTILISATEURS ET RÔLES

Auteurs de labels et opérateurs de politiques.

INTERFACES DANS LE CODE

buildEffectiveSelections ; validateSelections ; generateLabel ; LabelPreview.

DONNÉES ET ÉTATS

Sélections calculées, issues error/warning et LabelOutput.

DÉPENDANCES ET ÉCHANGES

Parseur SPIF, règles locales et backend de label.

COMPORTEMENT CIBLE PROPOSÉ

Frontends partage un contrat de validation versionné avec IAM ; le serveur Rust revalide chaque commande et trace le résultat.

MIGRATION DE CETTE CAPACITÉ

Comparer règles client et serveur sur incompatibilités, catégories implicites et cardinalités ; n'accepter aucune divergence silencieuse.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un conflit bloque l'aperçu sauvegardable ; contourner le JavaScript ne permet pas de persister un label invalide.

Références de l'analyse

[monorepo](#) · [services/datasphere-front/app/src/lib/spif-rules.ts:229](#) Validation des sélections.

[monorepo](#) · [services/datasphere-front/app/src/components/SpifSelector.tsx:121](#) Blocage de génération en cas d'erreur.

[monorepo](#) · [services/datasphere-front/app/src/components/LabelPreview.tsx:20](#) Sérialisation des catégories.

DSPF-04 **Cible non qualifiée**

Enregistrer la policy avec identité renouvelée et retour utilisateur

COMPORTEMENT OBSERVÉ

Le bouton de sauvegarde est désactivé sans UUID valide, politique ou classification. Il envoie un PUT de policy au data-model puis affiche succès ou erreur. Axios extrait `response.data` et un interceptor vérifie/renouvelle le token Keycloak avant d'ajouter le Bearer ; l'option `withCredentials` est configurée. Le message de succès correspond à la requête de sauvegarde, pas à la preuve qu'un export chiffré a été produit.

UTILISATEURS ET RÔLES

Utilisateurs authentifiés et opérateurs.

INTERFACES DANS LE CODE

PUT `/datasets/{datasetId}/label/policy` ; `saveDatasetPolicy` ; interceptor Axios.

DONNÉES ET ÉTATS

État `isSaving`, policy envoyée et retour toast ; token de session.

DÉPENDANCES ET ÉCHANGES

data-model, keycloak-js et configuration runtime.

COMPORTEMENT CIBLE PROPOSÉ

Frontend TS remplace l'adaptateur Keycloak par OIDC sans JVM et affiche le statut durable de génération de label.

MIGRATION DE CETTE CAPACITÉ

Préserver renouvellement, gestion d'erreur et accessibilité ; relier le toast à un reçu de commande plutôt qu'à une publication supposée.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un token expiré est renouvelé ou l'action refusée ; double-clic ne crée pas deux jobs ; succès de label et succès d'export restent distincts.

Références de l'analyse

[monorepo · services/datasphere-front/app/src/types/dataset.ts:23](#) Contrat de sauvegarde.

[monorepo · services/datasphere-front/app/src/config/axios.ts:10](#) Renouvellement et Bearer.

[monorepo · services/datasphere-front/app/src/components/LabelPreview.tsx:146](#) Conditions du bouton.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- Frontends TypeScript conserve ce parcours ; IAM/Catalog fournit une politique versionnée et une édition dataset explicite.
- Frontend TS conserve la composition ; la décision serveur se fonde sur la même version de politique et des identifiants stables.
- Frontends partage un contrat de validation versionné avec IAM ; le serveur Rust revalide chaque commande et trace le résultat.
- Frontend TS remplace l'adaptateur Keycloak par OIDC sans JVM et affiche le statut durable de génération de label.

Plan de données proposé

- Aucun moteur de données dans le navigateur ; lecture des politiques et commandes de sauvegarde autorisées.

Découpage du core Rust

- Aucun frontend Rust requis ; cœur serveur label/validation en Rust, contrat TS généré et adaptateur OIDC.

Identité, autorisation et isolation

- Le navigateur ne tranche ni clearance ni droit d'export ; serveur autoritaire.
- Validation et chargement XML ne doivent pas autoriser une URL interne arbitraire via le proxy.

Exploitation et reprise

- Application Vite/bun ; état local React et configuration runtime ; aucun test UI exécuté.

Séquence de migration

- Préserver formats XML et erreurs, migrer URLs backend et inclure version de politique dans la sauvegarde.
- Créer fixtures de politiques réelles anonymisées et comparer restauration des labels et changements de politique.
- Comparer règles client et serveur sur incompatibilités, catégories implicites et cardinalités ; n'accepter aucune divergence silencieuse.
- Préserver renouvellement, gestion d'erreur et accessibilité ; relier le toast à un reçu de commande plutôt qu'à une publication supposée.

Écarts et décisions encore ouverts

- L'UI présente des privacy marks, mais la policy envoyée contient policy/classification/category : caractériser leur représentation attendue dans le contrat serveur.
- Version de politique et édition dataset non explicites dans le parcours lu ; à ajouter.

Références de l'analyse

[monorepo](#) · [services/datasphere-front/app/src/App.tsx:9](#) Contexte dataset et chargement.

[monorepo](#) · [services/datasphere-front/app/src/components/SpifSelector.tsx:33](#) Restauration depuis le label courant.

[monorepo](#) · [services/datasphere-front/app/src/lib/spif-rules.ts:229](#) Validation des sélections.

[monorepo](#) · [services/datasphere-front/app/src/types/dataset.ts:23](#) Contrat de sauvegarde.